Agentic Framework

Agent Builder Architecture

Overview

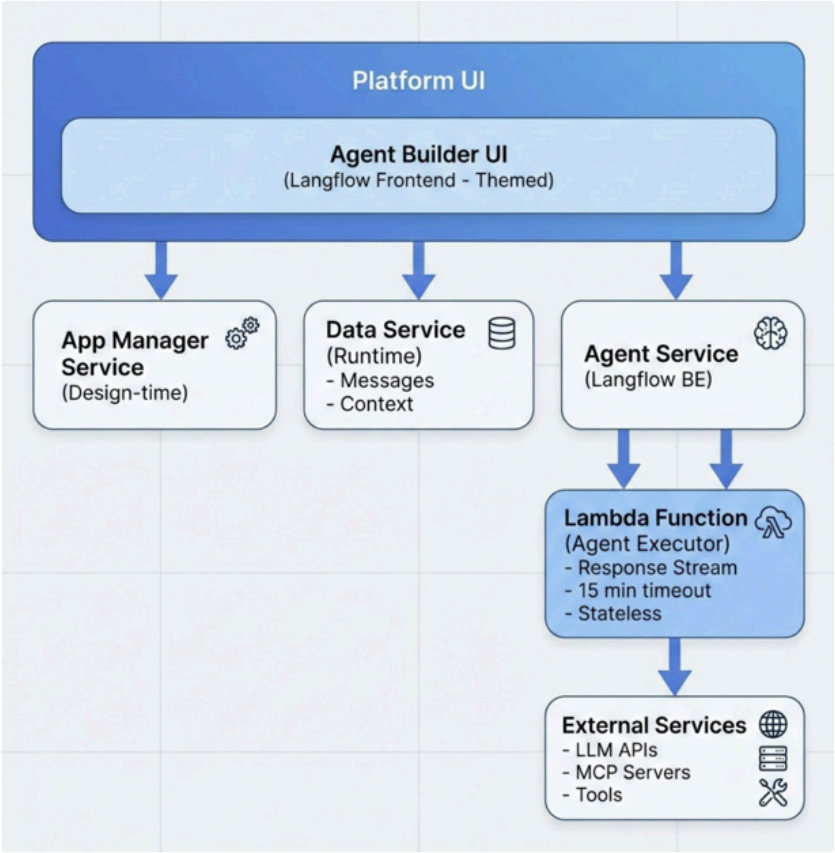
This document outlines the architecture for integrating Langflow into the platform as the Agent Builder module. The integration separates concerns between design-time operations (flow creation/editing), runtime operations (execution data), and agent orchestration.

Components

Component	Description	Technology
Platform UI	Main platform frontend application	React/TypeScript
Agent Builder UI	Langflow frontend (integrated into Platform UI)	React/TypeScript
App Manager Service	Handles design-time data (flows, nodes, configurations)	REST API
Data Service	Handles runtime data (execution logs, metrics, outputs, message history)	REST API

Agent Service	Langflow backend - agent building, orchestration, static data	FastAPI (Python)
Agent Executor (Lambda)	AWS Lambda for async agent execution with response streaming	AWS Lambda + Container Image

Component Diagram



UI Integration

Goals

- Seamless integration of Langflow frontend into Platform UI
- Consistent theming and UX across the platform
- Clear separation of API responsibilities

Implementation Details

Aspect	Details
Theme Integration	Replace Langflow theme system with Platform UI theme (colors, typography, components)
Routing	Mount Agent Builder routes under `/agent-builder/*` in Platform UI
Authentication	Use Platform UI's auth context; pass tokens to backend services
State Management	Integrate with Platform UI's global state where applicable

API Routing Strategy

Operation	Target Service	Examples
Flow CRUD	App Manager Service	Create/Read/Update/Delete flows
Node CRUD	App Manager Service	Add/modify/remove nodes in flows
Component metadata	App Manager Service	Node types, templates, configurations
Execution logs	Data Service	Run history, step outputs, errors
Metrics & analytics	Data Service	Performance data, usage statistics
Static data	Agent Service	Available node types, MCP services, connectors
Flow execution trigger	Agent Service	Start/stop agent runs

UI Migration Tasks

- Extract Langflow frontend components into Platform UI codebase
- Replace Langflow design system with Platform UI components
- Update API client layer to route to appropriate services
- Implement SSO/token passthrough for authentication
- Adapt React contexts for Platform UI integration

Backend Architecture

Agent Service (Langflow Backend)

Responsibilities:

- Agent/flow building and validation
- Orchestration of agent execution
- Serving static configuration data (node definitions, MCP services, etc.)
- Scheduling agent execution via Celery
- Managing connector configurations

Key Endpoints:

Endpoint	Method	Description
`/api/v1/nodes`	GET	List available node types
`/api/v1/mcp/services`	GET	List configured MCP services

`/api/v1/connectors`	GET	List available platform connectors
`/api/v1/flows/{id}/execute`	POST	Trigger flow execution
`/api/v1/flows/{id}/validate`	POST	Validate flow configuration

Integration Points:

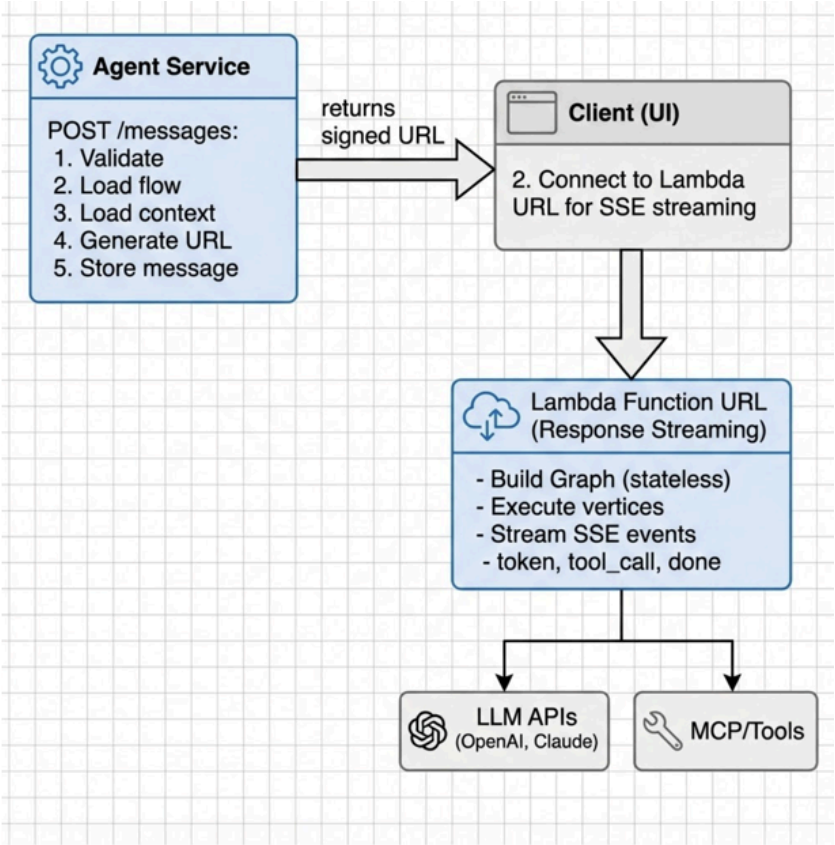
- All external connections (LLMs, databases, APIs) route through platform connectors
- MCP server configurations managed centrally
- Authentication via platform's identity service

Agent Executor (AWS Lambda)

Responsibilities:

- Execute agent flows asynchronously with response streaming
- Handle long-running tasks (up to 15 minutes) without blocking the API
- Auto-scale based on concurrent invocations
- Stream execution events directly to client via Lambda Response Streaming

Architecture:



Lambda Configuration:

Setting	Value	Description
Timeout	900s (15 min)	Maximum execution time
Memory	3008 MB	Max memory for better CPU
Package	Container Image	Required for large dependencies (LFX + Langflow)
Invoke Mode	RESPONSE_STREAM	Enables streaming responses

Auth Type	AWS_IAM	Signed URLs for security

Key Benefits:

- **Auto-scaling**: Handles burst traffic automatically (up to 1000 concurrent by default)
- **Cost-effective**: Pay only for execution time
- **Stateless**: No session state to manage, context loaded from Data Service
- **No infrastructure**: No Redis/RabbitMQ/ElastiCache to manage
- **Native streaming**: Direct client-to-Lambda SSE streaming
- **Managed**: AWS handles availability, patching, scaling
- **No VPC required**: Without ElastiCache, Lambda doesn't need VPC (faster cold starts)

Data Flow

Design-Time Flow (Creating/Editing Agents)

```
None
User Action → Agent Builder UI → App Manager Service → Database
                                        |
                                        ▼
                                Agent Service (validation)
```

Runtime Flow (Executing Agents)

1. `User sends message`

`  |`

`  ▼`

2. `Agent Service`

```
    │
    ├──► Load flow from App Manager Service
    ├──► Load conversation context from Data Service
    ├──► Store user message → Data Service
    │
    └──► Generate signed Lambda URL (with flow + context)
              │
              ▼
```

3. `Return signed Lambda URL to client`

```
              │
              ▼
```

4. `Client connects directly to Lambda Function URL`

```
              │
              ▼
```

5. `Lambda executes graph (stateless), streams SSE events to client`

```
    │
    ├──► LLM APIs (inference)
    └──► MCP/Tools (actions)
              │
              ▼
```

6. `On completion: Agent Service stores agent message → Data Service`

Security Considerations

- All inter-service communication authenticated via service tokens
- External connector credentials stored encrypted, never exposed to UI
- Agent execution sandboxed with resource limits
- Audit logging for all flow modifications and executions

Scalability

Component	Scaling Strategy
Agent Builder UI	Horizontal (CDN/load balancer)
Agent Service	Horizontal (stateless, load balanced)
Lambda Executor	Automatic (AWS manages based on concurrent invocations)
Data Service	Horizontal (stateless, load balanced)

Lambda Scaling Considerations

Aspect	Default	Can Increase
Concurrent executions	1,000 per region	Yes (request quota increase)
Burst concurrency	500-3000 (region dependent)	Limited
Provisioned concurrency	0 (cold starts)	Yes (eliminates cold starts)

Cold Start Mitigation:

- Use Provisioned Concurrency for production (keeps N instances warm)
- Container images have longer cold starts (5-10s) vs zip packages
- No VPC = faster cold starts (no ENI attachment delay)

End-to-End Agent Chat Flow

This section describes how chat messages flow through the system from user input to agent response.

Chat Flow Architecture

Flow Sequence (Happy Path)

Phase 1: Message Reception

1. User sends message via Chat UI
2. Request: `POST /api/v1/chat/{agent_id}/messages`

```
{
  "session_id": "sess_abc123",
  "content": "What's the weather in NYC?",
  "attachments": []
}
```

Phase 2: Agent Service Processing

3. Agent Service receives request
4. Validate session (create if new)
5. Load flow definition → App Manager Service (or cache)
6. Load conversation context → Data Service (message history for this session)
7. Store user message → Data Service

8. Generate signed Lambda Function URL:

```
lambda_url = generate_signed_function_url(
    function_name="agent-executor",
    payload={
        "flow_payload": flow.payload,
        "session_id": session_id,
        "job_id": job_id,
        "input_value": "What's the weather in NYC?",
        "context": conversation_history,  # Loaded from Data Service
    },
    expires_in=900  # 15 minutes
)
```

9. Return response to client:

```
{
  "session_id": "sess_abc123",
  "job_id": "job_xyz789",
  "stream_url": "https://xxx.lambda-url.us-east-1.on.aws/?payload=..."
}
```

Phase 3: Client Connects to Lambda

10. Client opens SSE connection to `stream_url`
11. Lambda function is invoked with response streaming enabled

Phase 4: Lambda Execution (Stateless)

12. Lambda parses flow payload + context from request
13. Builds Graph from payload: `Graph.from_payload(flow_payload)`
14. Injects conversation context into appropriate nodes (e.g., memory/chat nodes)
15. Executes graph with StreamingEventManager
16. For each vertex in topological order:
    - Execute vertex
    - Stream event: `on_end_vertex(vertex_id, outputs)`
    - If LLM node with streaming: stream `on_token` events for each chunk
    - If tool/MCP call: stream `on_tool_call` → execute → stream `on_tool_result`
    - Determine next vertices
17. On completion:
    - Stream `message` event with final response
    - Stream `done` event

Phase 5: SSE Stream (Direct from Lambda)

Lambda streams events directly to client:

```
event: token
data: {"content": "Let me "}

event: token
data: {"content": "check "}

event: tool_call
data: {"tool": "weather_api", "args": {"city": "NYC"}}

event: tool_result
```

`data: {"result": {"temp": 72, "condition": "sunny"}}`

`event: token`

`data: {"content": "The weather in NYC is 72°F and sunny"}`

`event: message`

`data: {"content": "The weather in NYC is 72°F and sunny", "job_id": "job_xyz789"}`

`event: done`

`data: {"job_id": "job_xyz789", "status": "ok"}`

Phase 6: Client Completion

18. Chat UI progressively renders:
    - Streaming text as tokens arrive
    - Tool call indicators (optional)
    - Final message when complete
19. On `done` event, client notifies Agent Service:
    - `POST /api/v1/chat/{agent_id}/messages/{job_id}/complete`
    - Agent Service stores agent message → Data Service

Event Types

Event	Description	Payload
`token`	LLM streaming chunk	`{content: string}`

`on_end_vertex`	Node execution completed	`{vertex_id, outputs, duration_ms}`
`tool_call`	Tool/MCP invocation started	`{tool: string, args: object}`
`tool_result`	Tool execution completed	`{tool: string, result: any}`
`message`	Final agent response	`{content: string, message_id: string}`
`error`	Execution error	`{error: string, vertex_id?: string}`
`done`	Stream complete	`{job_id: string, message_id: string}`

Session Management (Stateless)

Aspect	Details
Session ID	Client-provided or auto-generated UUID

State Storage	Data Service (no ElastiCache needed)
Message History	Persisted to Data Service database
Context Window	Configurable message limit for LLM context
Lambda	Stateless - context passed in each request

Stateless Execution Flow:

```
Turn 1: Agent Service loads context (empty) → Lambda executes → Agent Service stores message
Turn 2: Agent Service loads context (1 message) → Lambda executes → Agent Service stores message
Turn N: Agent Service loads context (N-1 messages) → Lambda executes → Agent Service stores message
```

Benefits of Stateless Design:

- No ElastiCache/Redis infrastructure needed
- No VPC configuration for Lambda (faster cold starts)
- Simpler architecture, easier debugging
- Context is always fresh from source of truth (Data Service)

Real-Time Communication Options

Protocol	Use Case	Pros	Cons
SSE (via Lambda)	Chat streaming	Direct client-Lambda, no relay needed	Unidirectional only
WebSocket	Voice mode, bidirectional	Full duplex, low latency	Requires API Gateway WebSocket

Recommendation: Use Lambda Response Streaming (SSE) for standard chat. Add WebSocket via API Gateway for voice/interactive modes if needed.

Design Decisions (Confirmed)

Decision	Choice	Rationale
Execution Runtime	AWS Lambda	Auto-scaling, pay-per-use, managed
Streaming Method	Lambda Response Streaming	Direct to client, no relay overhead
Session State	Stateless (Data Service)	No cache infrastructure, context from DB
Message Persistence	On completion only	Simpler, acceptable for rare failures
Flow + Context	In signed URL payload	Fresh data, no stale cache issues
Timeout	15 minutes	Matches Lambda max, sufficient for agents

VPC	Not required	No ElastiCache = faster cold starts